

1. FCFS (First Come First Serve) Scheduling

The simplest scheduling algorithm where processes are executed in the order they arrive.

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int bt[n], wt[n], tat[n];
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter Burst Time for each process:\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("P[%d]: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    wt[0] = 0; // First process doesn't wait
```

```
    for(i = 1; i < n; i++) {
```

```
        wt[i] = wt[i-1] + bt[i-1];
```

```
    }
```

```
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
```

```
    for(i = 0; i < n; i++) {
```

```

    tat[i] = bt[i] + wt[i];

    avg_wt += wt[i];

    avg_tat += tat[i];

    printf("P[%d]\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time: %.2f", avg_wt / n);

printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);

return 0;
}

```

2. Round Robin Scheduling

A preemptive algorithm where each process is assigned a fixed time slot (Quantum).

C

```

#include <stdio.h>

int main() {
    int n, i, qt, count = 0, temp, sq = 0;

    int bt[20], wt[20], tat[20], rem_bt[20];

    float awt = 0, att = 0;

    printf("Enter number of processes: ");

    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("P[%d] Burst Time: ", i + 1);

        scanf("%d", &bt[i]);
    }
}

```

```

    rem_bt[i] = bt[i];
}
printf("Enter Quantum Time: ");
scanf("%d", &qt);

while (1) {
    int done = 1;
    for (i = 0; i < n; i++) {
        if (rem_bt[i] > 0) {
            done = 0;
            if (rem_bt[i] > qt) {
                sq += qt;
                rem_bt[i] -= qt;
            } else {
                sq += rem_bt[i];
                wt[i] = sq - bt[i];
                rem_bt[i] = 0;
            }
        }
    }
    if (done == 1) break;
}

printf("\nProcess\tBurst\tWaiting\tTurnaround\n");
for (i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
}

```

```

    awt += wt[i];

    att += tat[i];

    printf("P[%d]\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAvg Waiting Time: %.2f\nAvg Turnaround Time: %.2f\n", awt/n, att/n);

return 0;
}

```

3. Banker's Algorithm (Deadlock Avoidance)

Determines if a system is in a safe state by simulating resource allocation.

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Processes: "); scanf("%d", &n);
```

```
    printf("Resources: "); scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], avail[m], need[n][m], f[n], ans[n], ind = 0;
```

```
    printf("Allocation Matrix:\n");
```

```
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
```

```
    printf("Max Matrix:\n");
```

```
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &max[i][j]);
```

```

printf("Available Resources: ");

for (i = 0; i < m; i++) scanf("%d", &avail[i]);


for (i = 0; i < n; i++) {
    f[i] = 0;

    for (j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j];
}


for (k = 0; k < n; k++) {
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {
            int flag = 0;

            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) { flag = 1; break; }
            }

            if (flag == 0) {
                ans[ind++] = i;

                for (int y = 0; y < m; y++) avail[y] += alloc[i][y];

                f[i] = 1;
            }
        }
    }
}


printf("\nSafe Sequence: ");

for (i = 0; i < n - 1; i++) printf(" P%d ->", ans[i]);

```

```
printf(" P%d\n", ans[n - 1]);  
return 0;  
}
```

4. Memory Allocation (First Fit & Best Fit)

Algorithms to determine where a process should be placed in memory.

First Fit

C

```
#include <stdio.h>
```

```
void main() {
```

```
    int bsize[10], psize[10], bno, pno, flags[10], allocation[10], i, j;
```

```
    for(i = 0; i < 10; i++) { flags[i] = 0; allocation[i] = -1; }
```

```
    printf("Enter no. of blocks: "); scanf("%d", &bno);
```

```
    printf("Block sizes: "); for(i = 0; i < bno; i++) scanf("%d", &bsize[i]);
```

```
    printf("Enter no. of processes: "); scanf("%d", &pno);
```

```
    printf("Process sizes: "); for(i = 0; i < pno; i++) scanf("%d", &psize[i]);
```

```
    for(i = 0; i < pno; i++)
```

```
        for(j = 0; j < bno; j++)
```

```
            if(flags[j] == 0 && bsize[j] >= psize[i]) {
```

```
                allocation[i] = j; flags[j] = 1; break;
```

```
            }
```

```
    printf("\nProcess No\tProcess Size\tBlock no.\n");
```

```

for(i = 0; i < pno; i++) {
    printf("%d\t\t%d\t\t", i+1, psize[i]);
    if(allocation[i] != -1) printf("%d\n", allocation[i] + 1);
    else printf("Not Allocated\n");
}
}

```

5. FIFO Page Replacement

Replaces the oldest page in memory when a new page is needed.

C

```
#include <stdio.h>
```

```

int main() {
    int i, j, n, a[50], frame[10], no, k, avail, count = 0;
    printf("Number of pages: "); scanf("%d", &n);
    printf("Page Reference String: "); for (i = 1; i <= n; i++) scanf("%d", &a[i]);
    printf("Number of frames: "); scanf("%d", &no);

    for (i = 0; i < no; i++) frame[i] = -1;
    j = 0;
    printf("\nRef String\tFrames\n");
    for (i = 1; i <= n; i++) {
        printf("%d\t\t", a[i]);
        avail = 0;
        for (k = 0; k < no; k++) if (frame[k] == a[i]) avail = 1;
        if (avail == 0) {

```

```

    frame[j] = a[i];
    j = (j + 1) % no;
    count++;
    for (k = 0; k < no; k++) printf("%d ", frame[k]);
}
printf("\n");
}
printf("Total Page Faults: %d\n", count);
return 0;
}

```

6. FCFS Disk Scheduling

Calculates total head movement of a disk arm based on request order.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int queue[20], n, head, i, seek = 0, diff;
```

```
    printf("Enter max range of disk: "); // e.g. 200
```

```
    int max; scanf("%d", &max);
```

```
    printf("Enter number of requests: "); scanf("%d", &n);
```

```
    printf("Enter requests: "); for(i = 0; i < n; i++) scanf("%d", &queue[i]);
```

```
    printf("Enter initial head position: "); scanf("%d", &head);
```

```
    for(i = 0; i < n; i++) {
```



```
    diff = abs(queue[i] - head);  
    seek += diff;  
    printf("Disk head moves from %d to %d with seek %d\n", head, queue[i], diff);  
    head = queue[i];  
}  
printf("Total seek time is %d\n", seek);  
return 0;  
}
```